Report

周奇

本文代码见于: Github, URL: `https://github.com/Tsianmy/AKNN`.

1 基础题

1.1 实现思路

输入数据分析

第一步是了解输入的数据格式，数据集 SIFT1M 和 GIST1M 来自 TEXMEX。从数据集官网可知数据的格式均为 $d, component$ 构成的向量，每一个向量为 $4 + 4d$ 字节。base 和 query 的维度为 128，可知每个点有 128 个分量。groundtruth 和另外两个数据的类型不一样，其中的向量为整型。且官网给出，groundtruth 的数据是对应每一个 query 的索引号（从零开始），索引号按距离从小到大排序。默认 $k = 100$，那么可以推测 $d = 100$，之后是 100 个索引号。

为了验证这一点，我先是计算 query 中的第 0 个点和 base 中前 100 个点的距离，然后计算第 0 个点和 groundtruth 中对应的 100 个临近点的距离。从结果看出，和 groundtruth 的点的距离确实比和 base 中前 100 个点的距离小，而且距离确实是从小到大。甚至，我将第 0 个点与所有 1M 个点的距离和第 0 个点与第 99 个（$k - 1$）临近点的距离进行比较，结果显示后者距离更小。

之后，用同样的方法测试了 KNN 图的数据，测试结果与推测相同， KNN 图的数据也是和 groundtruth 的数据格式相同。输入数据的格式整理成如 Table 1 所示。

Table 1. 输入数据格式

名称	数据类型	描述
d	int	向量维度
component	(unsigned char|float|int)*d	fvecs存点向量的分量，ivecs存近邻点的id

算法流程

算法 Algorithm 1 参考自 [1] 与 [2]，其中，$N(\mathbf{P}, E, G)$ 为 k-NN 图中点 $\mathbf{P}$ 的前 E 个最近邻点。

使用 C++ 实现并编译后，当 $E = 10, R = 1$ 时，查询 10000 次约耗 3 - 4 秒，查阅了已有的最近邻搜索 [3] 相关代码后发现，搜索时可以使用 Openmp 进行并行计算来加快搜索速度，于是采取并行查询，将搜索时间缩减到 1.1 秒。

Algorithm 1 Search$(G, \mathbf{q}, K, R, E, L)$

Input: k-NN 图 G，待查询点 $\mathbf{q}$，最近邻点数 K，重启次数 R，扩展结点数 E，候选集大小 L。

Output: Q 的 K 个最近邻点。

```
for r = 1, ..., R do
    i = 0，S = ∅
    p：G 中随机一点
    S = S ∪ {p}
    while i < L do
        i = S 中第一个未被检查过的下标
        标记 p_i
        for n in N(p_i, E, G) do
            If n 没有进入过候选集，S = S ∪ {n}
        end for
        在 S 中按与点 q 的距离大小进行排序
        保持 S 的容量不超过 L
    end while
    计算平均精度，选择平均精度大的搜索结果
end for
return S 中前 K 个点
```

数据结构

- 数据集可用动态数组存储。
- 候选集 S 可用 `std::vector` 表示。
- 计算准确率时，需要知道 groundtruth 中的近邻点，每个查询点的 k 近邻点可以存入以散列表为基础的 `std::unordered_set`，以提高查找效率。
- 可以用结构体存储点的索引和到查询点 $\mathbf{q}$ 的距离，然后重载比较运算符为按距离大小比较，使得对候选集排序后，距离小的在前。

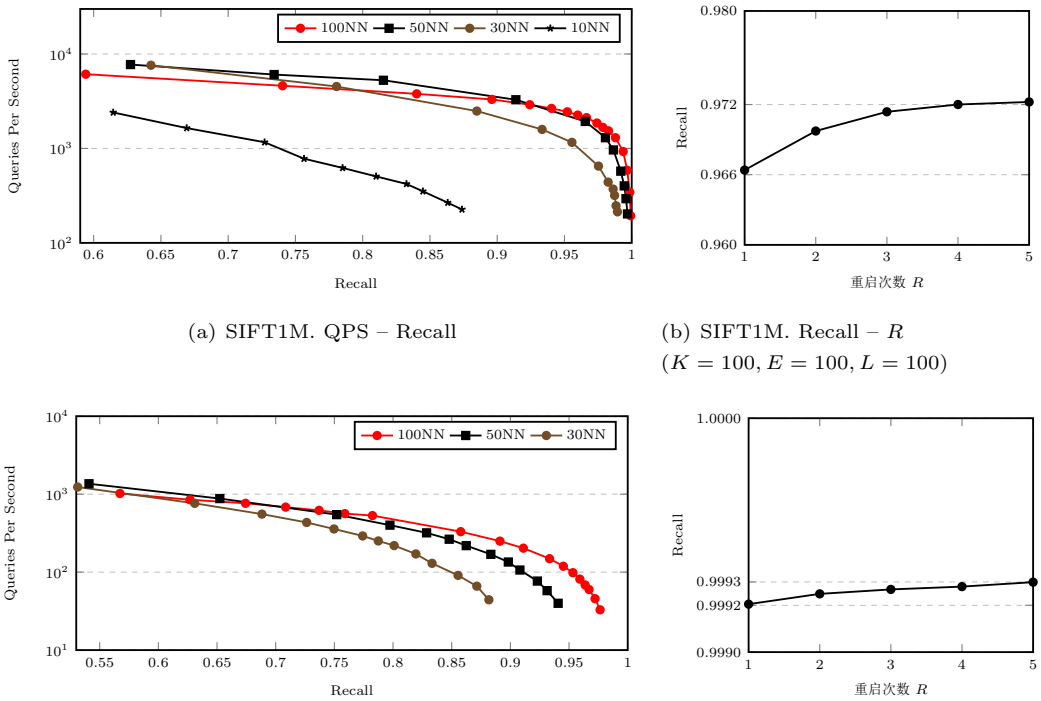
讨论

1. 优点
   - 实现简单，相比于 Brute Force ，LSH 等方法提升了计算速度。
2. 缺点
   - 数据集较大时增加了内存开销。
   - 精度受初始点影响，随机选择初始点使算法不稳定。
   - k 过于小时容易陷入局部最优。
3. 改进
   - 可以将整个图划分成若干个空间，将离平均坐标最近的点作为各空间初始候选点。每次搜索时，如果某一个候选点与待查询点 $\mathbf{q}$ 的距离小于和随机初始点的距

离，就从该候选点开始搜索，否则从随机初始点开始搜索。这样使得搜索从接近目标的区域开始，可以加快收敛速度。

- 缩小内存开销也是需要改进的方向。

1.2 结果评估



(a) SIFT1M. QPS – Recall

(b) SIFT1M. Recall – R ($K = 100, E = 100, L = 100$)

(c) GIST1M. QPS – Recall

(d) SIFT1M. Recall – R ($K = 100, E = 100, L = 1200$)

Figure 1. 搜索性能

搜索的结果如 Figure 1 所示，QPS 越小，精度越高。QPS 通过参数 E, L 来控制，先增加 E（10 - k），后增加 L（SIFT1M: 100 - 1200; GIST1M: 100 - 2500）。

图（a）和图（c）显示了不同的 k-NN 图上运行的效果，当 QPS 比较高时，k 稍小的图精度较高；当 QPS 减小到一定程度时由参数 L 主导，此时 k 越大的图越有利。

从图（b）和图（d）看出，重启次数对精度提升十分小，但搜索时间却成倍增加。

内存峰值

- SIFT1M：内存峰值为 0.95GB。
- GIST1M：内存峰值为 3.71GB。

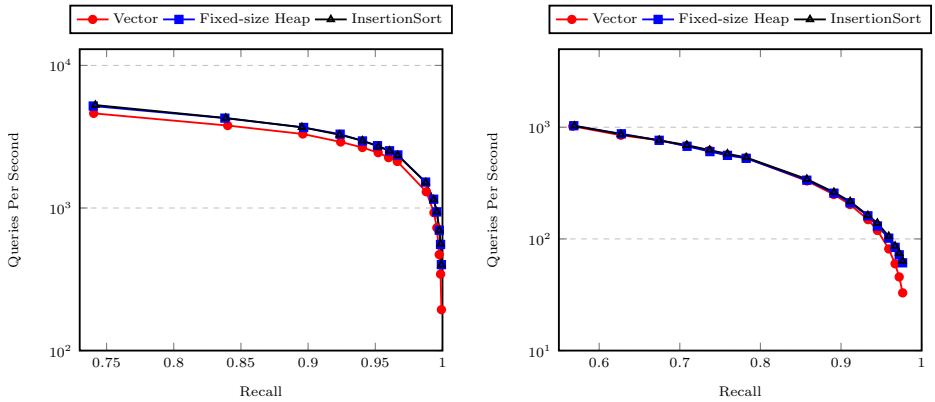
1.3 Candidate Pool 结构

固定大小的堆实现 可以用最大堆实现，堆的容量达到 L 时，扩展的点和堆顶元素比较，如果距离比堆顶小，弹出堆顶，扩展点进入堆，结束搜索后依次弹出，翻转次序即为按距

离从小到大的近邻点。也可以使用最小堆实现，找出叶子结点中距离最大的，如果扩展点距离比该结点小，就替换该结点，然后向上调整。本文用最小堆来实现 Candidate Pool。

插入排序实现 利用插入排序的思想，找到 Candidate Pool 中合适的位置进行插入，所有元素向后移动。

结果评估



(a) SIFT1M. QPS – Recall (b) GIST1M. QPS – Recall

Figure 2. 不同 Candidate Pool 结构的搜索性能

Figure 2 显示了在 100-NN 图上用不同结构实现 candidate pool 的搜索结果。从结果可以看出，使用堆结构或是插入排序可以提升搜索效率，堆结构和插入排序的效果相当。在 SIFT1M 上，效果提升明显；在 GIST1M 上，recall 较高时搜索效率有明显的提升。

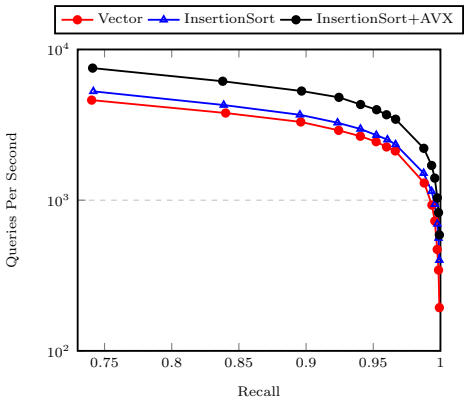
1.4 使用 SSE/AVX 指令集

使用 AVX 指令集可以同时对 8 个浮点数进行计算。以两个向量的每 8 个分量为一组进行计算，先相减再平方，得出结果加到临时向量的 8 个分量中。将临时向量的分量求和，再加上余下的不足 8 个的分量的差值的平方和，结果等于两个向量的距离的平方，也就是所有分量的差值的平方和。最后开方，即为欧式距离。

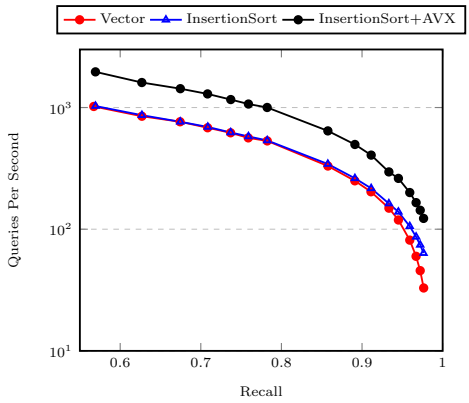
指令参考了 [4]，计算方法参考了 [5]。搜索的性能如 Figure 3 所示，从图中看出，AVX 指令集使得欧式距离的计算速度明显加快。

1.5 使用内存映射文件/mmap 技术

使用内存映射文件可以避免将所有数据载入内存。将文件映射到进程的内存空间，发生缺页中断时，才把数据换入内存。 Figure 4 显示了使用内存映射文件前后的搜索效果。由于 SIFT1M 的数据较小，一次性换入的数据较多，使得搜索速度立刻跟上不使用内存映射文件的速度。而在 GIST1M 上，数据被逐步换入，搜索速度缓慢上升。同时 Windows 和 Linux 下运行的结果略有不同，Linux 下一开始换入的数据似乎更多。

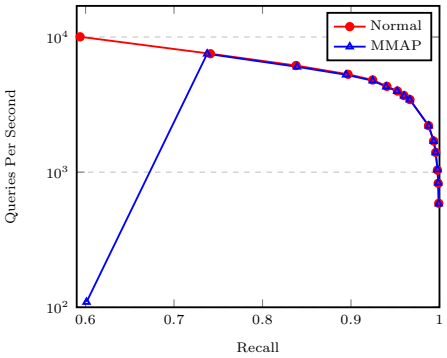


(a) SIFT1M. QPS – Recall

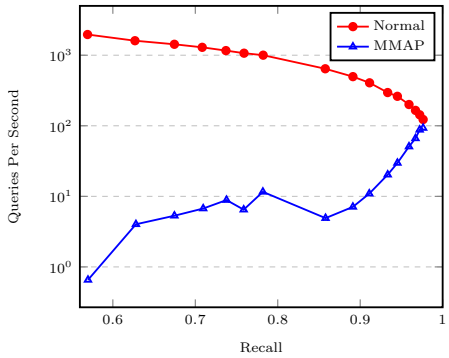


(b) GIST1M. QPS – Recall

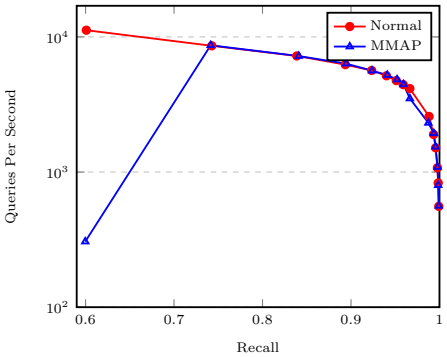
Figure 3. 使用 SSE/AVX 指令集优化欧氏距离计算性能



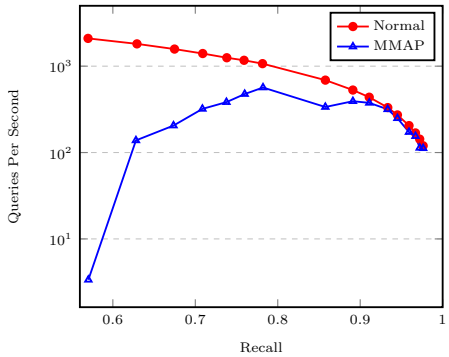
(a) SIFT1M. QPS – Recall on Windows



(b) GIST1M. QPS – Recall on Windows



(c) SIFT1M. QPS – Recall on Linux



(d) GIST1M. QPS – Recall on Linux

Figure 4. 使用内存映射技术

2 进阶题

2.1 调参过程

Product Quantization（向量量化）将向量空间分割为 M 个子空间，每个空间进行 k-means 聚类，用类中心点近似替代该类的所有点 [6]。因此，子空间数量 M 和类的数量 k

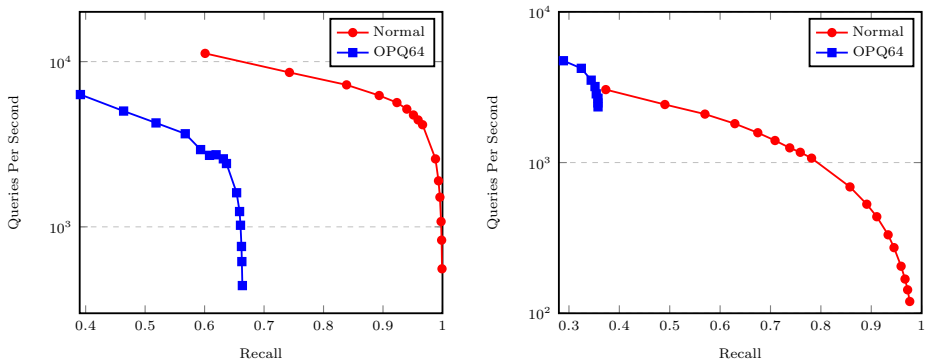
是人为给定且可以调整的参数。

k 可以看成一个码字能表示的最大范围，如果一个码字使用 1 个字节（8 位）的整数，k 最大可以是 256。k 越大，类的区域越小，类中心的误差越小，但是构建码表所需的空间和时间消耗越大。为了节省空间和时间，将码字设为一个字节，同时为了利用码字的表示范围，将 k 设为能够表示的最大值，即 256。

剩下需要调整的是 M 。为了使各个子空间的维度相等，将 M 设为向量总维度 d 的因数。因此可以设定 $M \in \{2^k | 2 <= 2^k <= d, k = 1, 2, ...\}$。使用网格搜索法得到，当 $M = 64$ 时，recall 最大。

2.2 结果评估

本题借助第三方代码 [7] 完成了码表的构建，搜索的性能如 Figure 5 所示。Product Quantization 的使用在很大程度上减少了内存的占用，但是搜索精度和速度随之下降，在 GIST1M 上表现的十分明显。



(a) SIFT1M. QPS – Recall

(b) GIST1M. QPS – Recall

Figure 5. 使用 Optimized Product Quantization

内存峰值

- SIFT1M：内存峰值为 0.49GB，基础题中内存峰值为 0.95GB。
- GIST1M：内存峰值为 0.42GB，基础题中内存峰值为 3.71GB。

Algorithm 2 Train

Input: Base 数据 $\mathbf{X}$，子空间数 M，一级量化聚类数 k^*。

Output: 类索引 index，类中心 centroids，码表 codes，β 的中心 Bcentroids。

```
nbits = 8
K-MEANS(k*, X)
根据类中心获取 index
计算 Bcentroids{β_i | β_i = X_i/q(X_i)}
Product Qiantization(B)
根据类中心获取 codes
return index, centroids, codes, Bcentroids
```

Algorithm 3 Search

Input: k-NN 图 G，待查询点 $\mathbf{q}$，最近邻点数 K，重启次数 R，扩展结点数 E，候选集大小 L，类索引 index，类中心 centroids，码表 codes，β 的中心 Bcentroids。

Output: Q 的 K 个最近邻点。

```
procedure L2-DISTANCE(a, q)
    从 index 获取 a 所属的类中心索引，从 centroids 获取类中心 c
    for m = 0 ... M - 1 do
        从 codes 获取 β 的索引，从 Bcentroids 获取 β
        a'(m * dsub : (m + 1) * dsub) ← β ⊗ c(m * dsub : (m + 1) * dsub)
    end for
    d = ||a' − q||^2
    return d
end procedure
for r = 1, ..., R do
    i = 0，S = ∅，p：G 中随机一点
    S = S ∪ {p}
    while i < L do
        i = S 中第一个未被检查过的下标
        for n in N(p_i, E, G) do
            距离 d = L2-DISTANCE(n, q)
            If n 没有进入过候选集，S = S ∪ {n}
        end for
        在 S 中按与点 q 的距离大小进行排序
        保持 S 的容量不超过 L
    end while
    计算平均精度，选择平均精度大的搜索结果
end for
return S 中前 K 个点
```

3 开放题 2：Graph-based 算法与 Quantization 的结合

[8] 提出利用当前点的近邻点进行回归来近似当前点，说明仅仅使用聚类仍有较大误差。原先的 Product Quantization 由于在每个子空间进行了聚类，那么子空间的所有点都由中心点近似替代，这导致这些点计算时采用同样的距离。为了突出子空间的每个点的特性，我想到可以采用类似 IVFADC（inverted file system with the asymmetric distance computation）[9] 的方法，先进行一级量化，然后计算残差，将残差进行 Product Quantization，查询时将查询点归类，计算查询点和中心的残差，用残差计算距离。然而，用该方法进行搜索后得到的 recall 仅为 **0.001**（$K = L = E = 100$）。于是我尝试在查询时不计算查询点的残差，而是通过码表和类中心近似计算 base 中的点，用类中心加上残差，然后计算距离，这时搜索后得到的 recall 为 **0.25**（$K = L = E = 100$），但是仍然不如直接使用 Product Quantization 的结果。

经过以上实验，只好考虑放弃残差的方法。那么是否能采取类似 [8] 中的采取系数的方法？考虑一个向量 $\mathbf{X} = (x_1, x_2, ..., x_d)$ ，设量化后的向量 $q(\mathbf{X}) = (x'_1, x'_2, ..., x'_d)$，那么目标函数是

$$\min_{\boldsymbol{\beta}} \|\mathbf{X} - \boldsymbol{\beta} \otimes q(\mathbf{X})\|^2$$

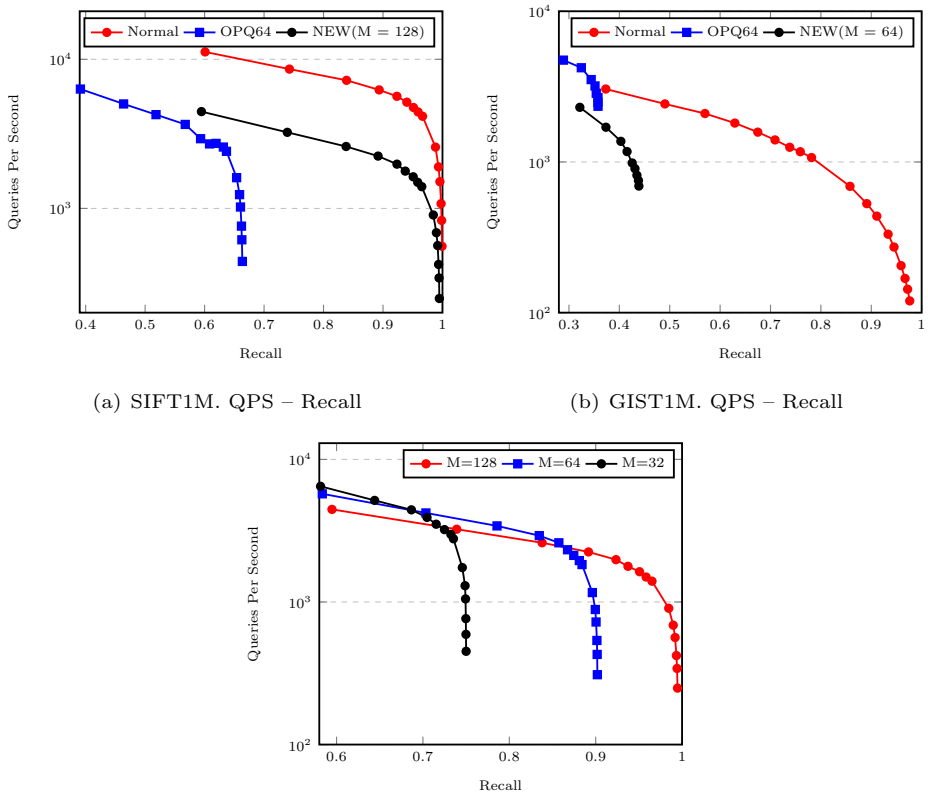
其中，定义 $\mathbf{a} \otimes \mathbf{b} = (a_1 b_1, a_2 b_2, ..., a_n b_n)$ 。

这里可以直接计算 $\beta_i = x_i / x'_i$，为了节省空间，对 $\boldsymbol{\beta}$ 进行 Product Quantization，保存码表待查询时使用，具体见 Algorithm 2 和 Algorithm 3。

在参数的调节过程中发现，一级量化的聚类数 k^* 越小，精度越高，当聚类的种类为 1 时，也就是第一次量化只计算所有点的平均值时，精度最高，所以将 k^* 设为 1。M 的调节是对搜索精度和空间时间的控制，M 越大，搜索精度越高，消耗的空间和时间也越多。

实验表明，使用该方法进行搜索的 recall 较高，见 3.1 节。

3.1 结果评估



(a) SIFT1M. QPS – Recall

(b) GIST1M. QPS – Recall

(c) SIFT1M. QPS – Recall

Figure 6. （a）（b）和其他方法的比较；（c）M 取不同值时使用新方法的搜索结果

从图上可以看出，新方法比直接使用 PQ 的精度要高，但是牺牲了图算法的一些速度。Table 2 展示了当 $K = L = E = 100$ 时各方法在 SIFT1M 上的搜索性能和内存占用。

内存峰值

- SIFT1M：内存峰值为 0.55GB，使用 OPQ 的内存峰值为 0.49GB，不使用 PQ 的内存峰值为 0.95GB。
- GIST1M：内存峰值为 0.45GB，使用 OPQ 的内存峰值为 0.42GB，不使用 PQ 的内存峰值为 3.71GB。

Table 2. 各方法在 SIFT1M 上的比较

Method	Memory(GB)	Recall($K = L = E = 100$)
Normal	0.95	0.966631
OPQ64	0.49	0.636791
New M=32	0.46	0.732358
New M=64	0.49	0.88418
New M=128	0.55	0.965194

References

[1] Kiana Hajebi et al. "Fast Approximate Nearest-Neighbor Search with k-Nearest Neighbor Graph". In: *IJCAI 2011, Proceedings of the 22nd International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain, July 16-22, 2011*. Ed. by Toby Walsh. IJCAI/AAAI, 2011, pp. 1312–1317.

[2] Cong Fu et al. "Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph". In: *Proc. VLDB Endow.* 12.5 (2019), pp. 461–474.

[3] *KGraph: A Library for Approximate Nearest Neighbor Search*. `https://github.com/aaalgo/kgraph`.

[4] *Intel@Intrinsics Guide*. `https://software.intel.com/sites/landingpage/IntrinsicsGuide`.

[5] 单线程、*SSE*、*AVX*运行效率对比——乘法累加运算. `https://blog.csdn.net/samylee/article/details/88944554`.

[6] Tiezheng Ge et al. "Optimized product quantization". In: *IEEE transactions on pattern analysis and machine intelligence* 36.4 (2013), pp. 744–755.

[7] *Faiss*. `https://github.com/facebookresearch/faiss`.

[8] Matthijs Douze, Alexandre Sablayrolles, and Hervé Jégou. "Link and code: Fast indexing with graphs and compact regression codes". In: *CVPR 2018 - IEEE Conference on Computer Vision & Pattern Recognition*. IEEE, 2018, pp. 3646–3654. DOI: `10.1109/CVPR.2018.00384`.

[9] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. "Product Quantization for Nearest Neighbor Search". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128. DOI: `10.1109/TPAMI.2010.57`.